

TECHNICAL ARCHITECTURE

NutriMart RAG

A deep dive into how this project works end to end: the problem it solves, the data it's built on, how a question travels through the system, and the design decisions behind each piece — including the ones that were tried and reverted.

LangChain

Hybrid Retrieval (BM25 + Dense)

Cross-Encoder Reranking

Groundedness Checking

FastAPI + React

Qdrant

Contents

1. What this is

2. Data

2.1 Structured store — SQLite (`db/products.sqlite`)

2.2 Vector store — Qdrant

3. Request flow

3.1 The deterministic fast path

3.2 LLM tool-calling router

4. Hybrid retrieval

5. Generation and groundedness

5.1 Typed-claim generation

5.2 Groundedness check

6. Conversation memory

7. LangChain integration

7.1 Gateway: multi-key rotation, proactive budgeting, fallback

8. Evaluation

9. Application-layer guardrails

10. Serving layer

11. Deployment

12. Design principles this project actually follows

A deep dive into how this project works end to end: the problem it solves, the data it's built on, how a question travels through the system, and the design decisions behind each piece — including the ones that were tried and reverted.

This document describes the LangChain-native pipeline only. Everything here is implemented with LangChain primitives (`ChatGroq`, `ChatHuggingFace`, LCEL chains, a custom `BaseRetriever`) rather than hand-rolled HTTP calls.

1. What this is

A RAG chatbot embedded in a mock quick-commerce app UI. It answers questions about 23 real packaged food products — ingredients, nutrition, allergens, and FSSAI (India's food regulator) compliance — by combining:

- **Deterministic structured data** (SQLite) for facts that have one correct answer: calories, ingredient lists, allergen declarations, pack size.
- **Retrieval-augmented generation** over a curated knowledge base (FSSAI additive/labelling rules, WHO nutrition guidance, an ingredient encyclopedia) for questions that need interpretation or regulatory context: "is this preservative safe?", "can this product claim to be a good source of protein?"

The goal wasn't to prove an LLM can answer these questions — a raw LLM call already can, badly, by hallucinating regulatory limits it was never given. The goal was building the retrieval, grounding, and verification machinery that makes those answers *trustworthy*: every claim is typed and cited, and a post-generation check verifies a cited number actually appears in the source it claims to come from.

2. Data

2.1 Structured store — SQLite (`db/products.sqlite`)

One row per product, loaded from `data/raw/products_compiled.json` by `ingestion/load_products.py`. Nested fields (nutrition, allergens, pack size) are stored as JSON text columns and queried with `json_extract()` — deliberately not normalized into separate

tables, since the dataset is only 23 products and normalization would add joins without adding value at this scale.

Fact-lookup questions ("how many calories does this have?") bypass retrieval and generation entirely — they're answered by a direct SQL query, with zero LLM calls. This is a real cost optimization: a large fraction of real questions about a product are simple fact lookups, and routing those around the LLM entirely means faster answers with no hallucination surface at all for that class of question.

2.2 Vector store — Qdrant

One flat collection (`kb_baseLine`), holding markdown chunks from four knowledge base files:

File	Content	Chunks
<code>fssai_knowledge_base.md</code>	Additive limits, labelling rules, marketing-claim eligibility	52
<code>nutrition_knowledge_base.md</code>	WHO Sugars 2015 + Healthy Diet guidance	15
<code>ingredient_knowledge_base.md</code>	Tier 1: INS-coded additives (aspartame, preservatives, colours...)	41
<code>ingredient_kb_tier2.md</code>	Tier 2: base/whole ingredients (wheat, milk, sugar...)	12

Chunked by `ingestion/parse_kb.py` on markdown `##` / `###` headers — no smart table handling, no overlap, no chunk-size tuning. This is intentional: the chunking strategy isn't the thing this project set out to demonstrate, so it stays simple rather than over-engineered.

Each chunk carries payload metadata when the source markdown declares it: `doc_type` , `entity` (which ingredient/additive it's about), `ins_no` (the regulatory code, which can be a **compound** value like `"627/631/635"` for a multi-code entry — every place this gets compared against a single code splits it first, since a naive string-equality check silently excludes every multi-code entry from retrieval otherwise), and `comparison_group` (an author-assigned tag linking chunks across files that answer a comparison question together, e.g. `sugar_vs_sweetener`).

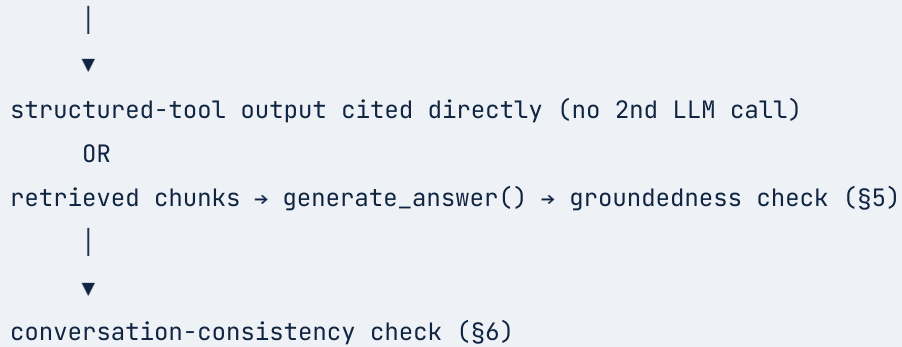
Embedded with `BAAI/bge-small-en-v1.5` . bge models require an instruction prefix on the **query** side only, never on documents — a documented model requirement (`BGE_QUERY_PREFIX` in

`config.py`), not a retrieval-strategy choice.

Metadata coverage is genuinely thin and uneven across the KB — not every chunk has a real `doc_type` / `entity` / `ins_no` . Every filtering/scoring mechanism downstream is deliberately **inclusive by default**: it only excludes a candidate when it has specific, positive evidence against it (e.g. "this chunk's `ins_no` doesn't match any code this product actually declares"), never on the absence of metadata.

3. Request flow





3.1 The deterministic fast path

`classify_query()` resolves which of the 23 catalog products (if any) a query is about by scoring name/brand token overlap — ties or a zero score return "no match" rather than guessing. If a product resolves **and** the query matches a known fact-field pattern **and** no regulatory-override term ("safe", "legal", "compare", "too much", etc.) is present, it routes straight to SQL with zero LLM involvement.

This fast path is deliberately narrow — a real cost optimization for confident, unambiguous cases only. Everything it doesn't confidently resolve falls through to the LLM tool-calling router below, not to a second keyword table. An earlier version of this project used a second layer of hand-written intent-classification keyword lists for the retrieval side; it was deleted outright after repeatedly hitting the same failure mode — a phrase list that didn't anticipate one new wording — which no amount of list-patching can close in general. LLM tool-calling replaced it because understanding "does this question need the knowledge base" is exactly the kind of open-ended language task an LLM is suited for and a keyword table isn't.

3.2 LLM tool-calling router

A single-round decision (not an open-ended agentic loop — every real routing failure found in development was fixable by one correct tool choice, none needed sequential tool chaining) with `tool_choice="required"` : the model must commit to at least one tool rather than declining. The system prompt instructs it to call a structured tool **and** `search_knowledge_base` together whenever a question needs both the product's own data and general regulatory/nutrition context (e.g. "is this healthy?", "why does this need a preservative?").

Updated 2026-08-30 (always-synthesize refactor): every structured-only turn now always goes through a second, real generation call (`generate_answer_lc()` , `verdict_mode=True`) instead of returning the raw tool string directly. The prior "no second LLM call" instant path was found to be the root cause of a whole recurring bug class: a regex vocabulary list decided whether a question

was "evaluative enough" to deserve real reasoning, and any real phrasing not on that list (a new health-judgment wording, a dietary question, a claim-eligibility question nobody had written a pattern for yet) fell through to a literal, unreasoned fact dump instead of an actual answer. Removing that gate — letting the LLM always reason over the real structured data it's handed, never inventing anything beyond it — closed that whole class at once rather than requiring one more regex entry per new phrasing. The only route that keeps the true zero-LLM-call "instant" property is `classify_query()`'s own deterministic SQL fast path (§3.1) — a single unambiguous fact lookup with no judgment involved.

LLM tool-selection has an irreducible error rate — this is treated as a property of the architecture, not a bug to chase to zero with more prompt engineering. Where a systematic gap was found (the model reliably under-calling `search_knowledge_base` for a specific, recurring question shape — evaluative/health-judgment questions, regulatory-limit questions, claim-eligibility questions, dietary/nutritional-verdict questions), the fix was a **structural pattern detector**: a regex that generalizes across phrasing, purely additive (it only ever adds more grounding to an answer, never blocks or overrides the model's own tool choices), never a hand-typed list of exact phrases. Four such detectors exist, covering health judgments, regulatory limits, claim eligibility, and dietary/nutritional verdicts — each was found via either a live user report or a proactive synthetic stress-test batch, and each is verified with regression tests. Since the always-synthesize refactor, these detectors' only remaining job is deciding whether to ALSO pull in KB retrieval alongside a structured tool's data — they no longer gate whether generation happens at all.

`classify_query()`'s own deterministic fast path (§3.1) has the same "regex vocabulary decides too much" shape at a layer *before* the LLM tool-router even runs, and needs the same category of fix whenever a new gap surfaces there (e.g. a question that mentions a nutrition-field word but is actually evaluative) — it's a smaller, shallower version of the same risk, not eliminated, and is a separate override-term list from the four detectors above by necessity (importing them here would pull the naive baseline's dependency-light fast path into the full hybrid stack's import graph).

4. Hybrid retrieval

When `search_knowledge_base` fires, the query goes through:

1. **BM25 + dense fusion** — BM25 (keyword/lexical) top-20 and dense (semantic/embedding) top-20 candidates, combined by Reciprocal Rank Fusion. Keyword and semantic search catch different failure modes: BM25 finds exact regulatory terms and product names dense search can miss; dense search finds paraphrases and conceptual matches BM25 can't.
2. **Narrow, evidence-based exclusion** — a candidate is dropped only when the query resolved to a specific product **and** the candidate is an ingredient chunk with a specific INS code that isn't among that product's actual declared codes. A functional-class boost (sourced from the knowledge base's own section headings, not a hand-typed list) also *excludes* a genuinely-declared ingredient chunk belonging to a different functional class than the one the query asked about (e.g. a query about "the colour additive" shouldn't be answered by a preservative chunk just because both are declared ingredients).
3. **Cross-encoder reranking** (`cross-encoder/ms-marco-MiniLM-L-6-v2`) of the top ~15 fused candidates, sigmoid-normalized.
4. **Corroborated-chunk trim** — a reranked chunk is kept only if it's independently confident on the cross-encoder alone, OR it's a top-2 pick by *either* the cross-encoder ranking or the raw BM25 ranking. This exists because the cross-encoder is real but demonstrably miscalibrated on this KB's long, multi-topic technical chunks — a genuinely-needed chunk was measured scoring as low as 0.055 while a topically-adjacent-but-wrong chunk scored 0.631. A single fixed score-floor trim was tried first and reverted after it dropped a chunk the model then hallucinated a citation for; the two-signal corroboration check is what actually shipped.
5. **Corrective retry** — if the top reranked score is below a threshold (0.3, validated against real score distributions, not guessed), the query is rewritten and steps 1–4 run once more.
6. **BM25-consensus skip** — the retry is skipped, and the original top-1 result trusted, when that result is *also* a landslide BM25-score winner ($\geq 1.4\times$ the runner-up, both scores real and substantial). This exists because the cross-encoder's absolute score genuinely can't always be trusted (see step 4) — for one specific real question, the correct chunk was the cross-encoder's own #1 pick and a landslide BM25 winner, yet still scored below the retry threshold; the "corrective" retry was actually making a correct answer worse by rewriting away from it.
7. **comparison_group override** — if both the original and retried query still fail to clear the threshold, but the candidate pool contains two or more chunks sharing a real, pre-authored `comparison_group` tag spanning different source files, that pairing is returned instead of an insufficient-evidence result. Deliberately narrow: it only fires on an explicit tag match, not general multi-hop synthesis. A genuinely untagged multi-chunk-synthesis question (no single chunk has the full answer, no tag exists) correctly falls through to insufficient-evidence — general sub-query decomposition for that class of question is a known, explicitly out-of-scope gap.

If nothing clears the bar even after all of the above, the pipeline returns an honest `[UNCERTAIN]` `Insufficient evidence...` message rather than guessing.

5. Generation and groundedness

5.1 Typed-claim generation

The system prompt (`generation/llm.py`) requires every non-trivial claim to be tagged `[FACT]` / `[REGULATORY]` / `[INTERPRETATION]` / `[DERIVED CALCULATION]` / `[UNCERTAIN]` , each with a source citation, and explicitly forbids filling retrieval gaps from the model's own outside knowledge. This matters most exactly where it's hardest to enforce: an additive whose retrieved context is a genuine, confirmed regulatory gap (the KB simply doesn't cover it) needs the model to say so, not to confidently invent a plausible-sounding limit.

5.2 Groundedness check

A post-generation, numeric-only verification: each `[FACT]` / `[REGULATORY]` / `[INTERPRETATION]` claim's cited number (if it has one) is checked against the actual text of the chunk it cites, and annotated with an inline `⚠️ [UNVERIFIED – ...]` marker on mismatch — never silently regenerated or edited.

Deliberately numeric-only, not general NLI. Three general entailment/contradiction-checking designs (whole-chunk entailment, per-sentence contradiction, whole-chunk contradiction) were built and tested against real chunks; all three produced real false positives on this KB's long, multi-topic technical chunks. A claim with no extractable number passes through unchecked — a known, stated limitation, not an oversight. This is a concrete instance of a broader theme in this project: several "obviously more thorough" designs were tried, measured against real data, and reverted in favor of a narrower mechanism that actually works reliably — see also the corrective-retry threshold (§4) and the groundedness scope itself.

6. Conversation memory

Follow-up questions ("is that too much?", "what about the diet version?") are expanded into self-contained queries *before* they reach routing — `conversation/resolve.py`, template-based, not an LLM call. This is a deliberate choice, for two reasons: it needs to keep working even when every configured LLM provider is exhausted (a real, recurring constraint during development — see §8), and template-based resolution is easier to verify and won't casually reintroduce the same product name into an already-scoped query in a way that skews retrieval toward product/ingredient chunks over the generic guidance chunks a question actually needs.

`conversation/state.py` tracks `known_facts` (attribute → value/unit/source, established across turns), the active product, and the active topic. Two checks build on this:

Added 2026-08-30: `conversation/state.py` also tracks a bounded `recent_turns` list (the last 3 real `(user_text, assistant_text)` pairs), threaded into the generation prompt as actual prior chat messages, not just flattened into `known_facts`. This is a distinct fix from the always-synthesize refactor above and was needed for a different reason: generation previously had zero memory of what it had actually *said* last turn (only the numeric facts it had established), so an explicit "do not repeat the previous information" instruction had nothing to check against. Bounded (not the full conversation) to keep prompt growth predictable; resets whenever the active product changes, for the same cross-product-contradiction reason `known_facts` already resets.

Separately, `hybrid_core.py::_is_gibberish()` and `_META_SYSTEM_RE` reject low-quality or adversarial input *before* routing/the tool loop even run: pure-noise input (empty/whitespace, no letters at all, a single character or case-alternating pair held down, a random low-vowel-ratio keyboard mash) and jailbreak/implementation-probing questions, respectively. These exist because `tool_choice="required"` forces the router to commit to SOME tool on every turn — without a pre-check, garbage input still got answered as if it were a real question about whatever product was in context.

- **Consistency check** (`generation/consistency.py`) — catches a freshly generated answer contradicting an already-established fact (a genuine numeric contradiction) or claiming a known attribute is "unavailable" when it was already given earlier in the conversation. A guideline number appearing near the same unit as the product's own known value (e.g. "≤50g added sugars" next to a product's real "47.4g") is deliberately *not* flagged — several real false-positive shapes of this exact kind were found and fixed via live testing, each locked in with a regression test.
- **Groundedness check** (§5.2) — checks a claim against its *cited chunk*.

These check two different things and are both run: a claim can be perfectly grounded in its cited source while still contradicting something the user was already told two turns ago.

7. LangChain integration

Everything an LLM makes a real decision in — tool-routing and final-answer generation — goes through LangChain-native call paths (`ChatGroq` , `ChatHuggingFace` , an LCEL prompt chain). Everything that *isn't* an LLM call — BM25 fusion, reciprocal rank fusion, cross-encoder reranking, the comparison-group tag match, structured SQL dispatch, conversation-state bookkeeping, the regex safety-nets from §3.2 — is plain Python rather than forced into a LangChain abstraction it doesn't naturally fit. LangChain has no primitive for "cross-encoder-rerank a fused RRF pool" or "does this query mention a health condition" — these are just Python, kept in `hybrid_core.py` and called directly from the LangChain entrypoint so the retrieval-decision logic exists in exactly one place.

A custom retriever/embeddings wrapper is used instead of `langchain_qdrant` 's default integration where the project's Qdrant payload shape (flat, metadata-scoped fields) doesn't match what that integration expects out of the box.

7.1 Gateway: multi-key rotation, proactive budgeting, fallback

`groq_gateway_invoke()` implements, LangChain-natively, a three-layer call strategy:

1. **Proactive budget check** — before attempting a Groq key, a per-key daily token-budget ledger (persisted to disk, survives process restarts) is checked against 95% of Groq's real 200,000 token/day per-key limit, using a request-size estimate. A key predicted to be too close to exhaustion is skipped *before* spending a real network round-trip finding out.
2. **Reactive rate-limit catch** — if a key is attempted and actually rate-limited anyway (quota spent by something outside this ledger's visibility), the next key is tried.
3. **Hugging Face fallback** — once every configured key is proactively-or-actually exhausted, falls back to a `ChatHuggingFace` model. Multiple Groq keys configured means real headroom before this fallback is ever needed in practice.

A real compatibility bug was found and fixed in this fallback path: the installed `langchain_huggingface` version rejects any non-empty `tool_choice` — including "auto" — unless exactly one tool is bound, and this pipeline's routing call always binds four. The fix passes `None` whenever more than one tool is given, letting the fallback model see every tool and decide freely rather than crashing.

A global, disk-persisted **exact-match cache** (LangChain's `SQLiteCache`) sits in front of every model call — an identical (prompt, model, params) tuple returns instantly with zero token spend and zero impact on the budget ledger, since a cache hit never actually reaches the provider.

8. Evaluation

Two parallel eval harnesses, both against the same 20-question RAG-eligible set (`eval/test_questions.py`), computing the same four metrics via a decompose-then-verify LLM-as-judge technique:

- **Faithfulness** — does the generated answer's content hold up against the retrieved context (KB chunks *and* any legitimately-cited structured facts)?
- **Answer relevancy** — does the answer actually address the question asked?
- **Context precision** — how much of what was retrieved was actually relevant?
- **Context recall** — did retrieval surface what a good reference answer would need?

`eval/ragas_metrics.py` + `eval/run_ragas_eval.py` — a lightweight, dependency-free implementation, routed through `generation/gateway.py::complete()` so every judge call gets the same multi-key rotation, proactive token-budget tracking, and HF fallback as normal pipeline traffic. This is the routine, cheap-to-run harness.

`eval/run_real_ragas.py` — the `ragas` PyPI package itself, as a second opinion from a community-maintained implementation. Getting it running against this stack required working around three real upstream bugs (documented in the script's own docstring: an eager, unused `ChatVertexAI` import broken against this project's `langchain-community` version; an outdated `instructor` pin; `ragas`'s own `provider="groq"` path calling an Anthropic-shaped client method) and adding the same multi-key rotation the other harness gets from the gateway (a single hardcoded key hit Groq's *daily* quota mid-run on live testing — the retry/backoff that handles a transient per-minute limit is useless against that, rotation is the real fix). A full run found a genuine measurement bug in the script itself: it only passed retrieved KB chunks as context to the judge, so every claim legitimately grounded in `products.sqlite` instead (a structured-tool prefix, or a `known_facts` value woven into the prose) was scored as unsupported — not a real faithfulness problem, a context the judge was never shown. Fixing that moved the measured faithfulness score from 0.712 to 0.941 on the same 18 questions, with the answers themselves unchanged — see `ragas_real_report.md` for the full run and its own Notes section.

Running these harnesses against real questions found and drove fixes for several real bugs during development — a regression in a query-rewriting tool schema, an ingredient-matching gap, three routing/tool-selection gaps, and a retrieval-filtering bug (the compound- `ins_no` issue mentioned in §2.2) that had been silently excluding every multi-code KB entry from retrieval. This is the pattern this project follows generally: eval isn't a report generated once at the end, it's a tool that actively surfaces where the system is actually wrong — including bugs in the eval tooling itself, not just the pipeline it's measuring.

Offline regression tests (no LLM/network cost — pure-function checks with fixture data, or LLM calls monkeypatched to cost nothing) exist per module and are run any time that module changes: retrieval fusion/exclusion logic, the consistency false-positive fixes, the token-budget ledger, the security guardrails below, and the custom RAGAS-style metrics' own scoring functions.

9. Application-layer guardrails

Scoped to `/api/chat` specifically, since that's the endpoint that spends real provider quota:

- **Rate limiting** — a sliding window, per-client-IP.
- **Query length bound** — enforced by the request schema.
- **Prompt-injection heuristic** — a narrow regex matching only near-unambiguous instruction-override attempts ("ignore all previous instructions", "reveal your system prompt").
Deliberately not a blunt keyword blocklist: this KB's own domain language legitimately contains phrases like "ignore the label" or "disregard the packaging claim" that a blocklist would false-positive on. Flags the response but never hard -blocks — the real structural defense against off-topic or fabricated answers is the typed-claim system prompt (§5.1) and the groundedness check (§5.2), neither of which this heuristic changes.

10. Serving layer

Two FastAPI apps front the same pipeline:

- **The main app** (port 8000) — serves the built React frontend and `/api/chat / /api/products` . Every chat request goes through the full routing → retrieval/tools → generation → groundedness → consistency flow described above. This is the one meant to be deployed publicly.
- **A secondary, API-only app** (port 8001) — no frontend, adds an SSE streaming endpoint and extra observability fields (which tools fired this turn, per-stage latency, real token usage) that the main app doesn't expose. Useful for inspecting what the pipeline actually did on a given question, or for a client that wants to render tokens as they arrive rather than waiting for the full answer.

Both share one cached bundle of loaded models (embedding model, cross -encoder, BM25 index, Qdrant client) — built once at process startup, not reloaded per request.

11. Deployment

Containerized as a multi-stage Docker build: the React frontend is built in a Node stage, then copied into a Python runtime stage alongside the source. `products.sqlite` is built at image-build time (deterministic, offline -derivable data — no network call needed during the build).

Qdrant is **not** embedded in the image — it's a separate, stateful service, pointed at via `QDRANT_URL / QDRANT_API_KEY` (a managed instance, e.g. Qdrant Cloud) rather than the bare `localhost:6333` local development uses. The one manual step a deploy needs that the container can't do for itself: running the ingestion script once against that managed instance before the app can answer anything, since embedding/upserting needs to happen exactly once per knowledge-base version, not on every container boot.

12. Design principles this project actually follows

A few threads run through the decisions above, worth naming directly:

- **Prefer the narrow, evidence-based mechanism over the broad, clever one.** The exclusion/boost logic in retrieval, the groundedness check's numeric-only scope, the comparison-group override's exact-tag requirement — each of these is a case where a more

general version was either tried and reverted (produced false positives on real data) or deliberately not built (the data didn't support it working reliably).

- **Cost-aware by default, not as an afterthought.** The deterministic fact-lookup fast path, the "structured-tools-only" instant path, the proactive token-budget ledger, and the exact-match LLM cache all exist because every unnecessary LLM call is real latency and real cost at scale — this shows up as an architectural pattern (bypass generation entirely when possible), not just an infrastructure detail.
- **An eval harness that's actually used, not just built.** Every metric in §8 has a real story of a bug it found and a fix it drove — the harness is treated as an active diagnostic tool run against real questions, not a one-time report.
- **Honest failure over confident guessing.** Insufficient retrieved evidence returns `[UNCERTAIN]`, not a plausible-sounding fabrication. A genuine regulatory gap in the source data is reported as a gap, not papered over. This is the single property the typed-claim system prompt and the groundedness check both exist to protect.